

catalogizer_integration

Catalogizer Deep Integration Analysis

Repository: <https://github.com/vasic-digital/Catalogizer>
Analysis Date: 2026-04-30
Analyst: Integration Agent
Commit: e1307bd (main)
Total Submodules: 39
Total Commits: 1,135

1. Complete Submodule Map

Source: `.gitmodules` (39 submodules total)

#	Name	Path	URL	Commit
1	WebSocket-Client-TS	WebSocket-Client-TS	git@github.com:vasic-digital/WebSocket-Client-TS.git	—
2	UI-Components-React	UI-Components-React	git@github.com:vasic-digital/UI-Components-React.git	—
3	Challenges	Challenges	git@github.com:vasic-digital/Challenges.git	4390
4	assets	Assets	git@github.com:vasic-digital/Assets.git	0dc5
5	Concurrency	Concurrency	git@github.com:vasic-digital/Concurrency.git	3337
6	Config	Config	git@github.com:vasic-digital/Config.git	cb14
7	Filesystem	Filesystem	git@github.com:vasic-digital/Filesystem.git	9580

#	Name	Path	URL	Com
8	Database	Database	git@github.com:vasic-digital/ Database.git	90f3
9	Auth	Auth	git@github.com:vasic-digital/ Auth.git	b4d6
10	Middleware	Middleware	git@github.com:vasic-digital/ Middleware.git	4e47
11	RateLimiter	RateLimiter	git@github.com:vasic-digital/ RateLimiter.git	—
12	Observability	Observability	git@github.com:vasic-digital/ Observability.git	—
13	Media	Media	git@github.com:vasic-digital/ Media.git	2fb2
14	Watcher	Watcher	git@github.com:vasic-digital/ Watcher.git	—
15	EventBus	EventBus	git@github.com:vasic-digital/ EventBus.git	0338
16	Cache	Cache	git@github.com:vasic-digital/ Cache.git	04eb
17	Security	Security		—

#	Name	Path	URL	Com
			git@github.com:vasic-digital/ Security.git	
18	Storage	Storage	git@github.com:vasic-digital/ Storage.git	—
19	Streaming	Streaming	git@github.com:vasic-digital/ Streaming.git	—
20	Discovery	Discovery	git@github.com:vasic-digital/ Discovery.git	8b77
21	Entities	Entities	git@github.com:vasic-digital/ Entities.git	f3dc
22	Media-Types-TS	Media-Types-TS	git@github.com:vasic-digital/ Media-Types-TS.git	1cf1
23	Catalogizer-API-Client-TS	Catalogizer-API-Client-TS	git@github.com:vasic-digital/ Catalogizer-API-Client-TS.git	9d12
24	Auth-Context-React	Auth-Context-React	git@github.com:vasic-digital/ Auth-Context-React.git	524c
25	Media-Browser-React	Media-Browser-React	git@github.com:vasic-digital/ Media-Browser-React.git	f31e
26	Dashboard-Analytics-React	Dashboard-Analytics-React	git@github.com:vasic-digital/ Dashboard-Analytics-React.git	af0a
27	Media-Player-React	Media-Player-React	git@github.com:vasic-digital/ Media-Player-React.git	0385
28	Collection-Manager-React	Collection-Manager-React	git@github.com:vasic-digital/ Collection-Manager-React.git	7789
29	containers	Containers	git@github.com:vasic-digital/ Containers.git	9f91

#	Name	Path	URL	Com
30	Lazy	Lazy	git@github.com:vasic-digital/ Lazy.git	a3d1
31	Memory	Memory	git@github.com:vasic-digital/ Memory.git	292a
32	Recovery	Recovery	git@github.com:vasic-digital/ Recovery.git	—
33	HelixQA	HelixQA	git@github.com:HelixDevelopment/ HelixQA.git	35de
34	DocProcessor	DocProcessor	git@github.com:HelixDevelopment/ DocProcessor.git	5f1e
35	LLMOrchestrator	LLMOrchestrator	git@github.com:HelixDevelopment/ LLMOrchestrator.git	1b95
36	LLMProvider	LLMProvider	git@github.com:HelixDevelopment/ LLMProvider.git	0720
37	VisionEngine	VisionEngine	git@github.com:HelixDevelopment/ VisionEngine.git	—
38	ScreenDiff	ScreenDiff	git@github.com:vasic-digital/ ScreenDiff.git	—
39	ReplayBuffer	ReplayBuffer	git@github.com:vasic-digital/ ReplayBuffer.git	—
40	VisualRegression	VisualRegression	git@github.com:vasic-digital/ VisualRegression.git	—
41	TrainingCollector	TrainingCollector	git@github.com:vasic-digital/ TrainingCollector.git	—

Key Observations: - 32 submodules are under vasic-digital org (reusable infrastructure) - 5 submodules are under HelixDevelopment org (AI/QA toolchain) - All Go modules are wired into catalog-api/go.mod via replace directives - All TypeScript/React packages

are linked via `package.json` dependencies - Submodule pointers are bumped in lockstep via automated cascades (commit messages like `governance: Article XI §11.9 user-mandate forensic anchor + cascade to 10 submodules`)

2. HelixQA Submodule Status

Current State in Catalogizer

- **Submodule path:** `HelixQA/`
- **Pinned commit:** `35deb43` (Phase 27.7 — `visionnav` Provider interface + `NopProvider`)
- **Latest upstream commit:** `0bca023` (Phase 29 — §12.6 Memory-Budget Ceiling)
- **Status:** Catalogizer is **~2 phases behind** latest HelixQA main
- **Last bump:** `e1307bd` — `chore(submodules): bump HelixQA to 35deb43` (Phase 27.7 `visionnav` Provider...)

HelixQA Repository Structure (upstream)

```
HelixQA/
├── banks/                # 60+ test banks (JSON/YAML)
├── challenges/           # Challenge definitions
├── cmd/                 # CLI binaries (helixqa-capture-demo, qa-audio-pro
├── data/                # Vector memory DB
├── docker/             # Container definitions
├── docs/
│   ├── benchmarks/
│   ├── diagrams/
│   ├── hooks/          # LD_PRELOAD shim guides
│   ├── nexus/
│   │   ├── ocu-roadmap.md    # OCU program status
│   │   ├── browser.md
│   │   ├── desktop.md
│   │   ├── mobile.md
│   │   ├── ai.md
│   │   └── ...
│   ├── openclawing/      # OpenClawing2 integration docs
│   ├── plans/
│   ├── releases/
│   ├── security/
│   ├── superpowers/     # OCU specs and plans
│   └── vision/
├── internal/visionserver/
├── monitoring/          # Campaign + dashboard
├── pkg/                 # Core Go packages
├── scripts/
├── tests/
├── tools/              # OSS vendoring
├── web/src/capture/     # Web capture frontend
└── website/            # Ticket viewer, challenges dashboard
```

How HelixQA is Referenced in Builds

- **No direct build dependency** in Catalogizer's Make/Build system for the submodule itself (it's a testing tool, not a runtime dependency)
 - **Integration points:**
 - `scripts/helixqa-orchestrator.sh` — orchestrates HelixQA autonomous sessions
 - `scripts/run-helixqa-api.sh`, `run-helixqa-web.sh`, `run-helixqa-desktop.sh` — bank runners
 - `docs/plans/2026-04-18-full-qa-cycle-master-plan.md` — mandates HelixQA as the **sole authorized UI automation tool**
 - `docker-compose.qa.yml` / `docker-compose.qa-robot.yml` — QA infrastructure containers
 - `catalog-api/handlers/challenge.go` — Challenge system integration
 - **Execution model:** HelixQA is cloned and run as an external tool; it exercises Catalogizer's running binaries via ADB (Android), CDP/Playwright (Web), chromedp/xdotool (Desktop), and HTTP (API)
-

3. Client Applications Map

3.1 Web Client

`catalog-web/` — Primary React web application - **Entry point:** `catalog-web/src/main.tsx` → `App.tsx` - **Build system:** Vite (`vite.config.ts`), Tailwind CSS, PostCSS - **Test framework:** Vitest (`vitest.config.ts`), Playwright E2E (`playwright.config.ts`, `e2e/`) - **Package manager:** npm (`package.json`, `package-lock.json`) - **Server:** Nginx (`nginx.conf`, `Dockerfile`) - **Architecture:** - `src/components/` — UI components - `src/pages/` — Route pages - `src/contexts/` — React contexts (auth, etc.) - `src/hooks/` — Custom hooks - `src/lib/` — Utilities, API client integration - `src/types/` — TypeScript types - `src/__tests__/` — Unit tests - `src/assets/` — Static assets - **API Connection:** Via Catalogizer-API-Client-TS submodule + `VITE_API_BASE_URL` env var - **WebSocket:** Via WebSocket-Client-TS submodule + `VITE_WS_URL` env var - **Submodules used:** Auth-Context-React, UI-Components-React, Media-Browser-React, Media-Player-React, Dashboard-Analytics-React, Collection-Manager-React, Media-Types-TS

3.2 Desktop Client

`catalogizer-desktop/` — Tauri-based cross-platform desktop app - **Entry point:** `src/main.tsx` → Tauri Rust backend (`src-tauri/`) - **Build system:** Vite frontend + Tauri 2.0 Rust backend - **Test framework:** Vitest (`vitest.config.ts`) - **Tech stack:** React 18 + TypeScript, TanStack Query, Zustand, Tailwind CSS - **Platforms:** Windows, macOS, Linux - **API Connection:** Same as web — HTTP REST + WebSocket via Catalogizer-API-Client-TS

`installer-wizard/` — Tauri-based desktop installation wizard - **Purpose:** Cross-platform SMB configuration wizard with network discovery - **Same tech stack** as `catalogizer-desktop` (Tauri 2.0 + React + Vite) - **Test coverage:** 93% claimed, 30/30 tests passing - **Build system:** Same Vite + Tauri pattern

3.3 Mobile Clients

catalogizer-android/ — Native Android phone app - **Entry point:** app/src/main/... (standard Android project structure) - **Build system:** Gradle (build.gradle.kts, settings.gradle.kts, gradlew) - **Tech stack:** Kotlin + Coroutines, Jetpack Compose, MVVM, Room, Retrofit, Hilt, Coil, ExoPlayer - **API Connection:** Retrofit HTTP client to catalog-api - **Last significant commit:** Fix 546 Android crashes, resolve 704 issues

catalogizer-androidtv/ — Android TV app - **Entry point:** app/src/main/... (standard Android TV project) - **Build system:** Gradle (same structure as phone app) - **Tech stack:** Kotlin + Coroutines, Jetpack Compose for TV (Leanback), MVVM, Room, Retrofit, Hilt, Coil - **API Connection:** Same Retrofit-based HTTP client - **Navigation:** D-pad optimized for big screen - **Special directories:** challenges/helixqa-banks/ — dedicated HelixQA validation bank

3.4 API Client Library

catalogizer-api-client/ — TypeScript client library - **Entry point:** src/index.ts (modular exports) - **Build system:** TypeScript compiler + bundler - **Test framework:** Vitest + Jest (dual config: vitest.config.ts, jest.config.js) - **Distribution:** npm package @catalogizer/api-client - **Output:** dist/ directory with compiled JS + type definitions - **Used by:** catalog-web, catalogizer-desktop, installer-wizard, and external consumers

3.5 CLI / Server Applications

catalog-api/ — Go REST API server (backend, not a client per se) - **Entry point:** cmd/boot/ (boot binary) - **No traditional CLI tool** beyond the server binary and migration tools

OCU-CUDA-Sidecar/ — GPU compute sidecar - **Entry point:** internal/server/ - **Protocol:** gRPC (proto/ definitions) - **Purpose:** Remote GPU dispatch for NVENC encoding, vision inference - **Deployment:** Docker container (Dockerfile)

Client Summary Table

Client	Platform	Tech Stack	Entry Point	Build Tool	Tests
catalog-web	Web (Browser)	React 18 + TS + Vite	src/main.tsx	Vite + Nginx	Vitest + Playwright E2E
catalogizer-desktop	Desktop (Win/Mac/Linux)	Tauri 2.0 (Rust) + React + TS	src/main.tsx + src-tauri/	Vite + Tauri CLI	Vitest
installer-wizard	Desktop (Win/Mac/Linux)	Tauri 2.0 (Rust) + React + TS	src/main.tsx + src-tauri/	Vite + Tauri CLI	Vitest
catalogizer-android	Mobile (Android)	Kotlin + Jetpack Compose	app/src/main/	Gradle	Android Instrumented
catalogizer-androidtv			app/src/main/	Gradle	Android Instrumented

Client	Platform	Tech Stack	Entry Point	Build Tool	Tests
	TV (Android TV)	Kotlin + Compose for TV			
catalogizer-api-client	Library (TS/JS)	TypeScript	src/ index.ts	tsc + bundler	Vitest + Jest
catalog-api	Server (Go)	Go 1.24 + Gin	cmd/boot/	go build	go test + race
OCU-CUDA-Sidecar	Sidecar (Go)	Go + gRPC + CUDA	internal/ server/	go build + Docker	go test

4. API/Services Architecture

4.1 High-Level Structure

Source: catalog-api/ARCHITECTURE.md

```

HTTP Request -> Gin Router -> Middleware (auth, CORS, metrics, rate limit)
|
Handler -> Service -> Repository -> database.DB (auto-rewrites SQL for
|
Media detection: filesystem.Client.ListDirectory() -> detector.Engine
|
Real-time: internal/media/realtime -> EventBus -> WebSocket -> connect

```

4.2 Key Technical Specifications

- **Framework:** Gin HTTP framework, routes under `/api/v1`
- **Database:** Dual-dialect — SQLite (dev) / PostgreSQL (production) with auto-rewriting SQL in `database/dialect.go`
- **Protocol support:** HTTP/3 (QUIC) via `quic-go/http3` + self-signed TLS; fallback HTTP/2 + gzip; Brotli compression via `andybalholm/brotli`
- **Dynamic port binding:** Writes `.service-port` for frontend discovery
- **22 submodule integrations** via `replace` directives in `go.mod`

4.3 Directory Structure

```

catalog-api/
├── cmd/boot/                # Boot binary entry point
├── handlers/                # Domain HTTP handlers
│   ├── admin_handler.go
│   ├── asset_handler.go
│   ├── auth_handler.go
│   ├── browse.go
│   ├── challenge.go        # Challenge system endpoint
│   ├── collection_handler.go
│   ├── configuration_handler.go
│   ├── conversion_handler.go
│   └── copy.go

```



```

— cover_handler.go
— download.go
— error_reporting_handler.go
— favorite_handler.go
— health_handler.go
— media_handler.go
— movie_handler.go
— music_handler.go
— playlist_handler.go
— search_handler.go
— smb_handler.go
— source_handler.go
— stream_handler.go
— system_handler.go
— tv_handler.go
— upload_handler.go
— user_handler.go
— services/                # Domain business logic
—   analytics_service.go
—   auth_service.go
—   challenge_service.go
—   configuration_service.go
—   configuration_wizard_service.go
—   conversion_service.go
—   error_reporting_service.go
—   favorite_service.go
—   media_service.go
—   movie_service.go
—   music_service.go
—   playlist_service.go
—   search_service.go
—   smb_service.go
—   source_service.go
—   stream_service.go
—   system_service.go
—   tv_service.go
—   user_service.go
— repository/              # Data access layer (CRUD)
— models/                  # Shared data structures
— database/                # Connection, dialect abstraction, migrations
— filesystem/              # Multi-protocol client (interface.go, factory)
— internal/
—   auth/                  # JWT authentication with role-based access
—   handlers/              # Infrastructure HTTP handlers
—   services/              # Aggregation, title parsing, scanning, media
—   middleware/            # Infrastructure middleware (auth, rate limit)
—   media/
—     detector/             # Rule-based media type detection
—     analyzer/            # Metadata extraction
—     providers/           # External metadata providers (TMDB, IMDB, Mus
—     realtime/            # Event bus -> WebSocket -> clients
—   smb/                   # Circuit breaker, offline cache, exponential
—   metrics/               # Prometheus metrics (/metrics endpoint)
—   lifecycle/             # LazyServiceRegistry for deferred service ini

```

		concurrency/	# Semaphore-based concurrency control
		httpclient/	# Pooled HTTP client with connection reuse and
		middleware/	# Domain middleware (CORS, logging)
		config/	# Configuration loading
		challenges/	# Challenge bank definitions, registration
		migrations/	# Database schema migrations
		scripts/	# Utility scripts
		tests/	# Test suites
		utils/	# Utilities

4.4 Media Detection Engine

Data Flow:

```

filesystem.Client.ListDirectory()
  -> detector.Engine (rule-based media type detection, 50+ types)
  -> analyzer (metadata extraction)
  -> provider.Registry (TMDB, IMDB, TVDB, MusicBrainz, Spotify, Steam, O
  -> entity creation -> hierarchy builder -> duplicate detection

```

Supported Media Types: Movies, TV shows/seasons/episodes, music (artists/albums/songs), games, software, documentaries, books, comics, and more.

4.5 External Metadata Providers

- TMDB (movies/TV)
- IMDB
- TVDB
- MusicBrainz
- Spotify
- Steam (games)
- OpenLibrary (books)

4.6 Real-time Updates

- **EventBus** submodule for typed pub/sub
- **WebSocket** hub with room/topic support (Streaming submodule)
- Connected clients receive live updates for downloads, scanning progress, source changes

5. Build System

5.1 Build Framework

Build/ directory contains a generic, reusable build framework: - **Semantic versioning** with build numbers via `versions.json` - **Change detection** using SHA256 source hashes (skips unchanged components) - **Container runtime detection** (Podman/Docker, rootless) - **Artifact generation** with `BUILD_INFO.json` and SHA256SUM checksums - **CLI interface** with `--dry-run`, `--force`, `--component`, `--bump` flags

5.2 Version Tracking

versions.json tracks all components:

```
{
  "schema_version": 1,
  "global": { "major": 2, "minor": 3, "patch": 0,
    "build_number": 25 },
  "components": {
    "catalog-api": { "last_build_number": 25, "last_build_date":
      "2026-04-28T23:35:43Z" },
    "catalog-web": { "last_build_number": 25, "last_build_date":
      "2026-04-28T23:36:38Z" },
    "catalogizer-desktop": { "last_build_number": 25, ... },
    "catalogizer-android": { "last_build_number": 25, ... },
    "catalogizer-androidtv": { "last_build_number": 25, ... },
    "installer-wizard": { "last_build_number": 25, ... },
    "catalogizer-api-client": { "last_build_number": 25, ... }
  }
}
```

5.3 Build Scripts

scripts/ directory: - build-all-releases.sh — Full release build for all platforms - build-test-release.sh — Test release build - container-build.sh — Container-based builds - ci-local.sh / ci-pipeline.sh — CI validation - helixqa-orchestrator.sh — HelixQA session orchestration - api-up.sh / api-down.sh — Service lifecycle - deploy.sh — Deployment script - distributed-boot.sh — Distributed system boot - audit-lazy-init.sh / audit-semaphores.sh — Audit scripts - gosec-scan.sh — Security scanning - devconnect.sh — Android device auto-connect - detect-landmines.sh — Pre-commit landmine detection - complete-remaining-tasks.sh — Task completion helper

Subdirectories: - scripts/android/ — Android-specific build scripts - scripts/audit/ — Audit automation - scripts/hooks/ — Git hooks + LLM-as-Judge pre-push gate - scripts/host_power_management/ — CONST-033 suspend guard - scripts/lib/ — Shared shell libraries - scripts/tests/ — Test runners

5.4 Docker / Containers

Docker Compose files: - docker-compose.yml — Main production stack - docker-compose.dev.yml — Development environment - docker-compose.dev.override.yml — Dev overrides - docker-compose.test.yml — Test infrastructure - docker-compose.test-infra.yml — Test services - docker-compose.qa.yml / docker-compose.qa-robot.yml — QA environment - docker-compose.security.yml — Security scanning - docker-compose.build.yml — Build environment

Services defined: PostgreSQL 15, Redis 7, catalogizer-server, transcoder, Nginx + React frontend, Prometheus, Grafana, backup service.

5.5 No CI/CD Pipelines (Constitution Mandate)

- **NO GitHub Actions** — `.github/workflows/` absent by design
 - **NO automated CI/CD** — All builds run manually or via Makefile/script targets
 - **NO Git hooks in production** — Pre-commit hooks exist but are optional/developer-only
 - This is a deliberate constitutional constraint (Universal Mandatory Constraints §1)
-

6. Existing QA Artifacts

6.1 Full-QA Master Plan

`docs/plans/2026-04-18-full-qa-cycle-master-plan.md` - **Governance:** CONSTITUTION.md Article VII (§7.1–§7.11) - **Session directory:** `docs/reports/qa-sessions/2026-04-18-T2158/` - **10-phase execution plan:** 1. Governance + plan + session directory 2. Clean rebuild (all apps + services) 3. Unit + integration tests (every submodule) 4. Challenges bank run 5. HelixQA bank tests (non-Android) 6. HelixQA autonomous QA (non-Android) 7. Video + screenshot post-session review 8. Fix loop (root-cause + regression tests) 9. Version bump + release artefacts 10. Final session report

6.2 QA Sessions Archive

`docs/reports/qa-sessions/` contains multiple sessions:

Session	Date	Contents
<code>2026-04-18-T2158/</code>	2026-04-18	Full-QA Master Cycle baseline (per plan)
<code>2026-04-21-T/</code>	2026-04-21	KeyPress Android-9 fallback, stagnation guard
<code>2026-04-21-T-v2/</code>	2026-04-21	Run5 ticket-persistence triage
<code>2026-04-22-T16-28/</code>	2026-04-22	Auto-committed session
<code>2026-04-22-T17-16/helixqa/</code>	2026-04-22	Android TV challenge validation closure

Each session directory contains: - `FINAL-REPORT.md` — Aggregated results - `logs/` — Per-run command logs - `challenges/` — JSON results + summary - `helixqa/` — Bank + autonomous results - `videos/` — MP4 recordings - `screenshots/` — Pre + post per action - `tickets/` — Markdown tickets with evidence - `analysis/` — Deep analysis, suggestions

6.3 Other QA-Related Reports

- `docs/reports/2026-04-21-session-closure-analysis.md` — DEFER-001 + DEFER-002 closure
- `docs/reports/2026-04-21-verification-plan.md` — Fuzz-found bug verification
- `docs/reports/COMPREHENSIVE_AUDIT_REPORT_2026-02-27.md`
- `docs/reports/TEST_COVERAGE_EXPANSION_PLAN.md`
- `docs/reports/SECURITY_SCAN_REPORT_2026-02-27.md`
- `docs/reports/MASTER-COMPLETION-2026-04-11.md`

6.4 Security Audits

- docs/audits/ — Regular audit reports
 - docs/reports/security/20260411-153546/ — Security scan baseline
 - Security tools: SonarQube, Snyk, Trivy, OWASP Dependency Check, gosec, Semgrep
 - Last audit: f9c7781 — docs(audit): final close — bank verification at 0 FAIL / 206 PASS / 1 SKIP
-

7. Testing Infrastructure

7.1 Go Backend Tests (catalog-api/)

- **Test framework:** testify + table-driven tests
- **Test helper:** In-memory SQLite via database.WrapDB(sqlDB, DialectSQLite)
- **Resource limits:** GOMAXPROCS=3 go test ./... -p 2 -parallel 2
- **Race detection:** -race flag mandatory
- **Constructor injection** for all services enables mock-based testing
- **Coverage files:** Multiple coverage.* files in services/ showing iterative coverage expansion
- **Handler tests:** Nearly every handler has a *_test.go file
- **Benchmark tests:** auth_service_bench_test.go
- **Dedicated challenge tests:** challenge_dedicated_test.go, challenge_handler_test.go

7.2 Web Frontend Tests (catalog-web/)

- **Unit tests:** Vitest (vitest.config.ts)
- **E2E tests:** Playwright (playwright.config.ts, e2e/)
- **Test setup:** src/test-setup.ts
- **Lighthouse:** lighthousec.json for performance auditing

7.3 Desktop Tests (catalogizer-desktop/, installer-wizard/)

- **Test framework:** Vitest (vitest.config.ts)
- **Tauri backend:** Rust tests in src-tauri/

7.4 Android Tests (catalogizer-android/, catalogizer-androidtv/)

- **Android instrumented tests** in app/src/androidTest/
- **Unit tests** in app/src/test/

7.5 API Client Tests (catalogizer-api-client/)

- **Dual framework:** Vitest + Jest (vitest.config.ts + jest.config.js)
- **Regression guards:** Permanent env-skip annotations

7.6 HelixQA Test Banks

HelixQA/banks/ contains 60+ test banks covering:

Category	Banks
Full-QA Campaigns	full-qa-api, full-qa-web, full-qa-android, full-qa-androidtv, full-qa-cross-platform
Nexus Platform-Specific	nexus-browser, nexus-desktop- {linux, macos, windows}, nexus-mobile- {android, ios}, nexus-ally, nexus-ai, nexus-perf, nexus-observability, nexus-xflow
OCU Program	ocu-foundation, ocu-capture, ocu-vision, ocu-interact, ocu-observe, ocu-record, ocu-automation, ocu-tickets, ocu-adversarial, ocu-cross-platform, ocu-fixes-validation
Fixes Validation	fixes-validation, fixes-validation-ally, fixes-validation-ai, fixes-validation-browser, fixes-validation-cover, fixes-validation-decoupling, fixes-validation-desktop
Feature-Specific	app-navigation, entity-management, file-browser, editor-operations, cloud-storage-operations, admin-operations
Security / Stress	ddos-ratelimit-comprehensive, edge-cases-stress, benchmarking-baselines
Capture / Input	capture-android, capture-linux, input-linux
Meta	docs-audit, image-quality-gate, atmosphere, cli-agents-comprehensive, aichat-bash-tools-comprehensive

7.7 Challenge System

- `catalog-api/challenges/` — Challenge bank definitions and registration
 - `catalog-api/handlers/challenge.go` — HTTP endpoint to run challenges
 - **Challenge execution:** Must be run through the `catalog-api` binary itself (no shell scripts/ `curl` per Constitution Article XI §11.3)
 - **Coverage goal:** Every feature has a registered challenge
-

8. Integration Gaps

8.1 HelixQA Version Gap

- **Gap:** Catalogizer pins HelixQA at `35deb43` (Phase 27.7); latest upstream is `0bca023` (Phase 29 — Memory-Budget Ceiling)
- **Impact:** Missing Phase 28 and Phase 29 improvements (memory budget enforcement, provider refinements)
- **Action:** Bump submodule pointer after compatibility verification

8.2 Android / Android TV Autonomous QA — FATAL BLOCKER

- **Status:** `ATMOSphere-only ADB devices` permanently excluded per `.devignore` (Constitution Article VII §7.1)
- **Impact:** `full-qa-android` and `full-qa-androidtv` banks are **SKIPPED** in autonomous mode
- **Unblocker:** Connect a non-ATMOSphere Android phone + Android TV via `adb connect`; add to `.devconnect`
- **Current workaround:** Manual APK install + manual testing; HelixQA banks exist but cannot run autonomously

8.3 CUDA Sidecar Gap

- **Status:** `OCU-CUDA-Sidecar/` exists but no CUDA sidecar is running on `thinker.local`
- **Impact:** Vision CUDA path + NVENC encode deferred; real OpenCV CUDA + TensorRT OCR not available
- **Unblocker:** Build + deploy `OCU-CUDA-Sidecar/` Docker image to `thinker.local`

8.4 Missing Platform Runners

- **iOS:** No iOS app exists; no macOS/Windows runners for native capture
- **Impact:** `nexus-mobile-ios` bank exists in HelixQA but has no target application

8.5 LD_PRELOAD Hook Compilation Gap

- **Status:** Hook-based observation requires per-target compiled shim
- **Impact:** `ocu-observe LD_PRELOAD` path not wired for production targets
- **Unblocker:** Operator must pick target + compile `docs/hooks/ld-preload-shim.c`

8.6 Coverage Gaps by Client

Client	Unit Tests	Integration	E2E	HelixQA Banks	Autonomous QA	Status
catalog-api	Yes	Yes	Yes	full-qa-api	Partial	Mostly covered
catalog-web	Yes	Yes	Yes (Playwright)	full-qa-web	Yes (chromedp)	Covered
catalogizer-desktop	Yes	Partial	No	nexus-desktop-*	Partial (stubbed)	Partial
installer-wizard	Yes	Partial	No	None identified	No	Gap
catalogizer-android	Yes	Yes	No	full-qa-android	Blocked	Gap
catalogizer-androidtv	Yes	Yes	No	full-qa-androidtv	Blocked	Gap
catalogizer-api-client	Yes	Yes	No	None identified	No	Gap
OCU-CUDA-Sidecar	Yes	No	No	None identified	No	Gap

8.7 Test-Only Code Concerns

- **Constitution Article IV** mandates universal solutions; no test-only code in apps
- **Constitution Article XI (Anti-Bluff)** requires every test to produce end-user-visible evidence
- **Risk:** Some tests may still be “bluff” tests (pass without exercising real behavior) — the Constitution explicitly calls out this historical problem
- **Mitigation:** Random audit required in every Full-QA Master Cycle (pick 5 tests + 5 challenges, comment out feature, confirm fail)

8.8 No Automated CI/CD

- **Constitution mandates NO CI/CD pipelines**
- **Impact:** All testing is manual or script-driven; no automated regression on every commit
- **Mitigation:** `scripts/ci-local.sh` and `scripts/ci-pipeline.sh` provide manual CI equivalents

9. OCU Program Status

9.1 Program Overview

OCU = OpenClaw Ultimate — 8 sub-projects (P0–P7) for autonomous multi-platform QA capture/interaction/observation/recording.

Source: `HelixQA/docs/nexus/ocu-roadmap.md`

9.2 Phase Status Table

Phase	Status	Closed Date	What Was Delivered
P0 Foundation	CLOSED	2026-04-17	Contracts + containers GPU extension + vertical-slice CLIs. All 10 P0 test categories green.
P1 Capture	CLOSED	2026-04-18	Factory + 4 CaptureSource backends (web/CDP, linux/X11, android, androidtv). Bank <code>ocu-capture.json</code> shipped.
P2 Vision	CLOSED	2026-04-18	Pipeline + CPU backend + remote-dispatch plumbing. Bank <code>ocu-vision.json</code> (13 entries) shipped.
P3 Interact	CLOSED	2026-04-18	Factory + 4 Interactor backends (linux/uinput, web/CDP, android, androidtv). Bank <code>ocu-interact.json</code> (19 entries) shipped.
P4 Observe	CLOSED	2026-04-18	Factory + 5 Observer backends (<code>ld_preload</code> , <code>plthook</code> , <code>dbus</code> , <code>cdp</code> , <code>ax_tree</code>). Bank <code>ocu-observe.json</code> (21 entries) shipped.
P5 Record	CLOSED	2026-04-18	Recorder + 3 encoder stubs (x264/nvenc/vaapi) + <code>FrameRing</code> + <code>clipper</code> . Bank <code>ocu-record.json</code> (21 entries) shipped.
P6 Automation	CLOSED	2026-04-18	Engine composes P1–P5 behind single <code>Perform()</code> . Bank <code>ocu-automation.json</code> (22 entries) shipped.
P7 Tickets+tests	CLOSED	2026-04-18	<code>pkg/ticket</code> : 12 <code>EvidenceKind</code> constants + <code>.ocu-replay</code> DSL. 4 cross-cutting banks (81 entries). v4.0.0 released.

9.3 Sub-Phase Closures (Wiring Real Backends)

Sub-Phase	Status	Notes
P1.5/P3.5 web+android wiring	CLOSED 2026-04-18	Production <code>chromedp</code> for web + ADB for android. 40 nexus packages -race green.
P1.5/P3.5 Linux wiring	CLOSED 2026-04-18	Production <code>xwd+convert</code> + <code>xdotool/xdotool</code> . 44 nexus packages -race green.

Sub-Phase	Status	Notes
P2.5 CPU vision real Diff+Analyze	CLOSED 2026-04-18	Pure-Go per-pixel diff + Sobel edge detection. No CGO.
P4.5 D-Bus + CDP observers	CLOSED 2026-04-18	godbus/dbus/v5 + chromedp ListenTarget.
P4.5 AT-SPI observer	CLOSED 2026-04-18	AT-SPI2 bus subscription for all events.
P4.5 LD_PRELOAD loader	CLOSED 2026-04-18	Shim path resolution + FIFO + JSON Lines parsing.
P5.5 x264 encoder via ffmpeg	CLOSED 2026-04-18	ffmpeg -c:v libx264 pipeline. 3 new tests.
P5.5 VA-API encoder via ffmpeg	CLOSED 2026-04-18	ffmpeg -c:v h264_vaapi with device node auto-detection.
P5.5 NVENC encoder via remote dispatch	CLOSED 2026-04-18	Lazy Worker resolution via contracts.Dispatcher. 6 tests.

9.4 Remaining Deferred Work

Item	Status	Deferred To
Real OpenCV CUDA + TensorRT OCR	Stubbed	P2.5 (LocalBackend interface preserves path)
Real /dev/uinput path	Stubbed	P3.5 (xdotool covers 95% of interactions)
FFmpeg NVENC CGO	Stubbed	P5.6 (operator scope on thinker.local)
Real WHIP/ WebRTC publisher	Stubbed	P5.5 (WebRTC/WHIP publisher off by default)
Real gRPC sidecar on thinker.local	Not deployed	P5.6 (OCU-CUDA-Sidecar exists but not running)
MKV clipper format	Deferred	P5.5 (ND-JSON only; MKV deferred)

9.5 Contract Versions (Locked)

Contract	Version
capture.go	v1
vision.go	v1
interact.go	v1
observe.go	v1

Contract	Version
record.go	v1
remote.go	v1

9.6 Release

- **OCU v4.0.0** released 2026-04-18
- **Release notes:** `HelixQA/docs/releases/v4.0.0.md`
- **All P0–P7 phases and sub-phases CLOSED**
- **Program is materially complete;** remaining items are operator-deployment scoped (CUDA sidecar on `thinker.local`)

10. Summary & Recommendations

Critical Findings

1. **HelixQA is 2 phases behind upstream** — bump to `0bca023` to get Memory-Budget Ceiling and latest provider refinements
2. **Android/Android TV autonomous QA is completely blocked** by the ATMOSphere device exclusion — this is the largest coverage gap
3. **OCU-CUDA-Sidecar is not deployed** on `thinker.local` — NVENC encoding and CUDA vision paths are stubbed
4. **installer-wizard and catalogizer-api-client have no HelixQA bank coverage** — they rely on manual testing only
5. **No CI/CD exists by constitutional mandate** — all QA is manual/script-driven, which scales poorly

Priority Actions

Priority	Action	Owner
P0	Bump HelixQA submodule to latest (<code>0bca023</code>)	Dev
P0	Connect non-ATMOSphere Android devices and unblock autonomous QA	Operator
P1	Build + deploy OCU-CUDA-Sidecar to <code>thinker.local</code>	Operator
P1	Add HelixQA banks for <code>installer-wizard</code> and <code>catalogizer-api-client</code>	QA/Dev
P2	Run Full-QA Master Cycle (Article VII) to completion with all 10 phases	QA
P2		QA

Priority	Action	Owner
	Audit 5 random tests + 5 challenges for bluff detection (Article XI)	

End of Report